ПОЛНЫЙ ОФЛАЙН-СПРАВОЧНИК

DX Framework

Движок, ECS, физика, NPC, корабль и развитие живого космического мира

DX_FrameWork

Версия 1.0 · 16 августа 2026

Содержание

- 1. Как пользоваться этим справочником без интернета
- 2. Что уже умеет проект
- 3. Сборка и запуск
- 4. Карта папок и ответственность кода
- 5. Минимум C++, необходимый для этого проекта
- 6. ECS с нуля
- 7. Один кадр игры и почему порядок критичен
- 8. Transform, координаты и математика
- 9. Сцена, импорт и сохранение
- 10. Рендер DirectX 11 простыми словами
- 11. Bullet Physics и типы тел
- 12. Капсульный контроллер игрока и NPC
- 13. Игрок: ввод, движение, камера и параметры
- 14. Взаимодействия и события
- 15. Анимация и визуальная модель
- 16. NPC AI: слои и поток решений
- 17. NavMesh, комнаты и двери
- 18. Как добавить новую задачу NPC
- 19. Энергетика
- 20. Комнаты, атмосфера, давление и затопление
- 21. Повреждения, оружие, разрезание и ремонт
- 22. Корабль, плавучесть, давление корпуса и соединения
- 23. Частицы, взрывы и эффекты
- 24. Редактор, UI и диагностика
- 25. Безопасный рецепт добавления новой функции
- 26. Диагностика: от симптома к причине
- 27. Производительность без преждевременной оптимизации
- 28. Save game и персистентный мир
- 29. Архитектура живого мира
- 30. Правила качества проекта
- 31. Что развивать следующим
- Приложение A. Краткая карта компонентов
- Приложение B. Чек-лист перед поездкой
- Приложение C. Шаблон записи о баге
- Приложение D. Словарь

1. Как пользоваться этим справочником без интернета

Не пытайся читать весь документ подряд. Используй его как карту.

1. Если забыл, **где находится код**, открой раздел 4 и приложение A.
2. Если не понимаешь, **почему система обновляется именно так**, открой разделы 5–7.
3. Если добавляешь механику, сначала прочитай раздел 25 «Безопасный рецепт новой функции».
4. Если что-то трясётся, проваливается, не строит путь или не сохраняется, начни с раздела 26.
5. Перед большим изменением сделай отдельный коммит или копию рабочей ветки.

Полезные соседние документы:

- `docs/CODE_METHOD_MAP.md` — подробный индекс существующих файлов и методов;
- `docs/CPP_ENGINE_THEORY_OFFLINE.md` — расширенная теория C++ и устройства движка;
- `docs/NPC_AI_DEBUG_GUIDE_RU.md` — диагностика NPC и NavMesh;
- `docs/PARTICLES_EXPLOSIONS_DECOMPRESSION_GUIDE_RU.md` — эффекты, взрывы и разгерметизация;
- `docs/MeshCuttingGuide.md` — разрезание и восстановление мешей;
- `docs/ROADMAP_14_DAYS_OFFLINE_STUDY_RU.md` — отдельная двухнедельная программа.

Правило работы после перерыва

Сначала ответь на четыре вопроса:

- Какие данные хранит функция? Это **компонент**.
- Кто меняет эти данные? Это **система**.
- Когда она запускается в кадре? Это **порядок обновления**.
- Кто владеет памятью и внешним ресурсом? Это **время жизни**.

Если ответы ясны, большая часть кода снова становится понятной.

2. Что уже умеет проект

Проект — не пустой каркас. В нём уже есть связанные вертикальные подсистемы:

- ECS: сущности, компоненты и системы;
- DirectX 11: модели, материалы, текстуры, шейдеры, свет, тени, skybox;
- Bullet Physics: rigid body, коллизии, лучи и капсульный контроллер персонажа;
- редактор на ImGui, выбор сущностей и изменение компонентов;
- импорт корабля из GLB, комнаты, двери и связи помещений;

- игрок, камера, здоровье, выносливость, кислород и взаимодействие;
- NPC: команды, utility AI, состояния, NavMesh, двери, следование и ремонт;
- энергетика: генераторы, потребители, батарея, кислородный генератор;
- повреждения, оружие, пробоины, разрезание меша и ремонт;
- давление, затопление, декомпрессия, плавучесть и физика корабля;
- частицы, взрывы, отладочная визуализация;
- сохранение и загрузка сцены в JSON;
- JobSystem для тяжёлых фоновых операций;
- асинхронная загрузка статических GLB и поэтапный импорт 250-МБ корабля с панелью прогресса.

Архитектурная цель

Любая новая механика должна быть частью симуляции, а не сценарием «для игрока».

Плохой вариант: когда игрок входит в комнату, появляется поломка. Хороший вариант: оборудование изнашивается, сеть перегружается, возникает поломка; NPC могут заметить её и устранить, даже если игрок далеко.

3. Сборка и запуск

Что нужно

- Windows x64;
- Visual Studio 2022 с workload **Desktop development with C++**;
- Windows SDK;
- зависимости проекта, уже настроенные в решении: DirectX, Bullet, Recast/Detour, Assimp, ImGui, JSON и вспомогательные библиотеки;
- решение `C:\Users\Lakis\Desktop\DX_FrameWork\DX_FrameWork.sln`.

Обычная сборка

1. Открой `DX_FrameWork.sln`.
2. Выбери конфигурацию `Debug` и платформу `x64`.
3. Нажми **Build → Build Solution**.
4. Исполняемый файл появляется в `C:\Users\Lakis\Desktop\DX_FrameWork\x64\Debug\DX_FrameWork.exe`.

Сборка из Developer PowerShell

```
msbuild "C:\Users\Lakis\Desktop\DX_FrameWork\DX_FrameWork.sln" `
```

```
/m /p:Configuration=Debug /p:Platform=x64
```

Если после изменения заголовка странные ошибки

1. Закрой запущенную игру.
2. Выполни **Build → Clean Solution**.
3. Выполни **Build → Rebuild Solution**.
4. Смотри первую ошибку компилятора, а не последнюю: остальные часто являются её следствием.

Минимальная проверка после сборки

- сцена загружается;
- `F5` переключает Editor/Play;
- игрок ходит, бежит, прыгает и спокойно стоит на полу;
- NPC следует за игроком и проходит низкий порог;
- дверь открывается;
- нет новых сообщений `assert`, `NaN`, `invalid entity` или `Detour could not project`.

4. Карта папок и ответственность кода

Папка	Что в ней искать
`src/Core`	приложение, главный кадр, время, файлы, физический мир
`src/ECS`	registry, базовые компоненты, физические и тестовые системы
`src/Gameplay/Player`	ввод игрока, движение, контроллер, facing, vitals
`src/Gameplay/AI`	команды NPC, utility, NavMesh, данные мозга
`src/Gameplay/Power`	генерация, потребители, батарея, кислород
`src/Gameplay/Combat`	оружие и нанесение повреждений
`src/Gameplay/Repair`	ремонт объектов, пробоин и runtime-меша
`src/Gameplay/Effects`	частицы, взрывы, декомпрессия
`src/Scene`	импорт, создание сущностей, JSON save/load, комнаты
`src/Render`	DirectX 11, модели, материалы, шейдеры, тени
`src/Editor`	панели ImGui, инспектор, отладочные кнопки
`assets`	модели, текстуры, шейдеры и данные сцены

Папка	Что в ней искать
`docs`	офлайн-теория, инструкции и планы

Главные точки входа

- `src/Core/ApplicationClass.cpp` — инициализация и порядок одного кадра;
- `src/ECS/ECSRegistry.h` — создание сущностей и работа с компонентами;
- `src/ECS/Components/Components.h` — большая часть базовых компонентов;
- `src/Scene/SceneManager.cpp` — создание, импорт, сохранение и загрузка мира;
- `src/Editor/EditorUI.cpp` — редактор и диагностические панели;
- `src/Gameplay/Player/CharacterControllerSystem.h` — общая капсульная физика игрока и NPC;
- `src/ECS/System/PhysicsSystem.cpp` — синхронизация Bullet и ECS;
- `src/Gameplay/AI/NpcNavMeshSystem.cpp` — построение и запросы путей.

5. Минимум C++, необходимый для этого проекта

Заголовок и `.cpp`

В `.h` обычно находится объявление: что существует. В `.cpp` — реализация: как оно работает.

```
// TemperatureSystem.h
#pragma once

class TemperatureSystem
{
public:
    void Update(float dt);
};
```

```
// TemperatureSystem.cpp
#include "TemperatureSystem.h"

void TemperatureSystem::Update(float dt)
{
    // Реальная логика.
}
```

Значение, ссылка и указатель

```
TransformComponent copy = original;       // отдельная копия
TransformComponent& ref = original;        // ссылка на тот же объект
TransformComponent* ptr = &original;       // указатель, может быть nullptr
```

В ECS `GetComponent<T>()` возвращает указатель, потому что компонент может отсутствовать.

```
if (auto* transform = registry.GetComponent<TransformComponent>(entity))
{
    transform->position.y += 1.0f;
}
```

Никогда не разыменовывай результат без проверки:

```
// Опасно: падение, если компонента нет.
registry.GetComponent<TransformComponent>(entity)->position.y = 0.0f;
```

`const`

`const` означает: через это имя данные не меняются.

```
const auto* room = registry.GetComponent<RoomComponent>(roomEntity);
```

Это полезно для читаемости и защиты от случайных изменений.

RAII и владение

Ресурс должен освобождаться владельцем автоматически. Для обычных объектов используй `std::unique_ptr`. Для Bullet-ресурсов следуй уже принятому в `RigidBodyComponent` и `CharacterController` правилу владения.

```
auto system = std::make_unique<TemperatureSystem>();
```

`RigidBodyComponent` и `CharacterController` являются move-only: их нельзя случайно копировать вместе с сырыми указателями Bullet.

```
RigidBodyComponent body;
registry.AddComponent<RigidBodyComponent>(entity, std::move(body));
```

Шаблоны

`AddComponent<T>` — шаблон: один алгоритм работает для разных типов компонентов.

```
registry.AddComponent<PlayerComponent>(entity, PlayerComponent{});
registry.AddComponent<TransformComponent>(entity, TransformComponent{});
```

Лямбда

Лямбда — функция без отдельного имени. В проекте она часто используется при обходе ECS.

```
registry.ForEach<TransformComponent, PlayerComponent>(
    [](Entity entity, TransformComponent& transform, PlayerComponent& player)
    {
        player.Normalize();
    });
```

Частые ошибки C++

- включение `.cpp` вместо `.h`;
- забытый `#include`;
- использование уничтоженной сущности;
- хранение указателя на элемент `unordered_map` после операции, которая могла изменить контейнер;
- копирование объекта, который владеет Bullet/D3D-ресурсом;
- изменение контейнера прямо во время обхода;
- несогласованность объявления в `.h` и определения в `.cpp`.

6. ECS с нуля

Что такое сущность

`Entity` — идентификатор. Он сам ничего не умеет. Значение сущности появляется из набора компонентов.

Например, кислородный генератор может иметь:

- `TransformComponent` — где находится;
- `NameComponent` — как называется;
- `OxygenGeneratorComponent` — как производит кислород;
- `PowerConsumerComponent` — сколько энергии просит;
- `RepairableComponent` — как ломается и ремонтируется;
- `InteractionComponent` — что делает клавиша взаимодействия.

Компонент

Компонент — только данные. Он не должен превращаться в большой объект с половиной логики игры.

```
struct HeatComponent
{
    float temperatureC = 20.0f;
    float heatCapacity = 100.0f;
    float dissipationPerSecond = 1.0f;
};
```

Система

Система находит подходящие сущности и меняет их данные.

```
class HeatSystem
{
public:
    void Update(ECSRegistry& registry, float dt)
    {
        registry.ForEach<HeatComponent>(
            [dt](Entity, HeatComponent& heat)
            {
                const float delta = heat.temperatureC - 20.0f;
                heat.temperatureC -= delta * heat.dissipationPerSecond * dt;
            });
    }
};
```

Полный жизненный цикл сущности

```
Entity generator = registry.CreateEntity();

registry.AddComponent<NameComponent>(
    generator, NameComponent{ "Aux Oxygen Generator" });

registry.AddComponent<TransformComponent>(
    generator, TransformComponent{});
```

```
registry.AddComponent<OxygenGeneratorComponent>(
    generator, OxygenGeneratorComponent{});

if (auto* oxygen = registry.GetComponent<OxygenGeneratorComponent>(generator))
{
    oxygen->enabled = true;
}

registry.RemoveComponent<OxygenGeneratorComponent>(generator);
registry.DestroyEntity(generator);
```

Теги

Пустой или почти пустой компонент может обозначать роль.

```
struct FireHazardTag {};
```

Проверка тега дешевле и яснее, чем сравнение имени объекта со строкой.

Важные правила ECS проекта

1. Не считай, что компонент существует: проверяй указатель.
2. Не уничтожай сущность внутри небезопасного обхода. Сначала собери ID в вектор.
3. Система не должна тайно менять несвязанные подсистемы.
4. Межсистемные факты передавай через компонент состояния или событие.
5. Entity ID не является вечным сохранённым указателем: при загрузке нужен remap ссылок.
6. `ActiveComponent` определяет участие многих объектов в обновлении и рендере.

Безопасное удаление после обхода

```
std::vector<Entity> toDestroy;

registry.ForEach<LifetimeComponent>(
    [&](Entity entity, LifetimeComponent& life)
    {
        life.remaining -= dt;
        if (life.remaining <= 0.0f)
            toDestroy.push_back(entity);
    });

for (Entity entity : toDestroy)
    registry.DestroyEntity(entity);
```

7. Один кадр игры и почему порядок критичен

Центр кадра — `ApplicationClass::Frame(float dt)`.

Упрощённый порядок:

1. ввод, горячие клавиши, состояние Editor/Play;
2. UI и управление курсором;
3. взаимодействие, двери, оружие, ремонт;
4. игрок формирует `CharacterMovement`;
5. NPC utility выбирает задачу;

6. NPC state выполняет задачу и формирует движение;
7. facing поворачивает персонажей;
8. character controller передаёт движение в Bullet;
9. animation выбирает и проигрывает клип;
10. повреждения и эффекты;
11. системы комнат, корабля, затопления, давления и физики;
12. после физики камера читает окончательный transform игрока;
13. обновление сцены, света и матриц;
14. редакторская/физическая диагностика;
15. рендер.

Почему нельзя менять порядок наугад

Если камера обновится до физики, она увидит прошлую позицию. Если анимация прочитает движение до того, как AI его сформировал, она опоздает на кадр. Если controller обновится до StateSystem, NPC не получит новую скорость.

`dt` и fixed step

`dt` кадра меняется: 0.012, 0.016, 0.021 секунды. Физика должна шагать стабильным интервалом, например 1/60 секунды.

```
accumulator += frameDt;

while (accumulator >= fixedDt)
{
    PhysicsFixedUpdate(fixedDt);
    accumulator -= fixedDt;
}

const float alpha = accumulator / fixedDt;
RenderInterpolate(alpha);
```

Логика, которая зависит от реального времени, умножает скорость на `dt`. Логика физического шага использует `fixedDt`.

8. Transform, координаты и математика

Проект использует трёхмерные координаты и `DirectXMath`. Ось `Y` — вверх.

```
XMFLOAT3 position{ 0.0f, 1.0f, 0.0f };
XMFLOAT3 rotation{ 0.0f, 90.0f, 0.0f };
XMFLOAT3 scale{ 1.0f, 1.0f, 1.0f };
```

Всегда выясняй, хранится поворот в градусах или радианах. Функции `XMMatrixRotation...` обычно ожидают радианы.

```
const float yawRadians = XMConvertToRadians(transform.rotation.y);
```

Локальное и мировое пространство

- локальная позиция — относительно родителя;
- мировая позиция — итоговая позиция в сцене;
- направление камеры нужно переводить в горизонтальную плоскость для ходьбы;
- нормаль поверхности — единичный вектор, перпендикулярный поверхности.

Интерполяция

```
XMFLOAT3 Lerp(const XMFLOAT3& a, const XMFLOAT3& b, float t)
{
    return {
        a.x + (b.x - a.x) * t,
        a.y + (b.y - a.y) * t,
        a.z + (b.z - a.z) * t
    };
}
```

`t` обычно ограничивают диапазоном `[0, 1]`.

9. Сцена, импорт и сохранение

`SceneManager` отвечает за сущности сцены, импорт GLB, комнаты, свет, сериализацию и загрузку.

Основные операции:

```
Entity entity = sceneManager.SpawnObject(std::move(desc));
bool saved = sceneManager.SaveScene("Level.json");
bool loaded = sceneManager.LoadScene("Level.json", device, context);
```

`SpawnObjectDesc`

Используй descriptor, когда создаётся объект с моделью, transform и параметрами. Это лучше, чем функция с десятью позиционными аргументами.

```
SpawnObjectDesc desc;
desc.position = { 2.0f, 0.0f, 5.0f };
// Заполни модель, имя и другие поля, определённые в Components.h.
Entity object = sceneManager.SpawnObject(std::move(desc));
```

Entity remap при загрузке

Сохранённый ID может отличаться от нового ID после загрузки. Поэтому ссылки вида «дверь → комната», «пробоина → панель», «свет → объект» должны пройти через таблицу remap.

```
std::unordered_map<uint32_t, Entity> remap;
// oldId -> newEntity
```

После создания всех сущностей вызывается восстановление межсущностных ссылок.

Как добавить сериализацию нового компонента

Допустим, добавлен `HeatComponent`.

Сохранение:

```
if (const auto* heat = registry.GetComponent<HeatComponent>(entity))
{
    objectJson["heat"] = {
        { "temperatureC", heat->temperatureC },
        { "heatCapacity", heat->heatCapacity },
        { "dissipationPerSecond", heat->dissipationPerSecond }
    };
}
```

Загрузка:

```
if (objectJson.contains("heat"))
{
    const auto& value = objectJson["heat"];
    HeatComponent heat;
    heat.temperatureC = value.value("temperatureC", 20.0f);
    heat.heatCapacity = value.value("heatCapacity", 100.0f);
    heat.dissipationPerSecond = value.value("dissipationPerSecond", 1.0f);
    registry.AddComponent<HeatComponent>(entity, heat);
}
```

Всегда задавай безопасные значения по умолчанию — старые сцены не содержат нового поля.

Контрольный список сериализации

- компонент сохраняется;
- компонент загружается;
- отсутствующие поля имеют default;
- ссылки на Entity проходят remap;
- runtime-указатели Bullet/D3D не пишутся в JSON;
- после load внешние ресурсы пересоздаются;
- старый `Level.json` всё ещё загружается.

Асинхронная загрузка GLB и корабля

Тяжёлый `SpaceShip.glb` занимает около 250 МБ. Старый путь делал подряд на главном потоке всё: читал файл, разбирал GLB через Assimp, обходил все узлы, преобразовывал вершины, создавал Direct3D-буферы, материалы, ECS-сущности и физические тела. Пока эта цепочка не закончилась, игровой цикл не мог нарисовать следующий кадр и Windows считал окно зависшим.

Асинхронность в движке теперь разделена на две разные задачи:

Что загружается	CPU-часть	Установка результата	Материалы
Обычная статическая модель, включая `.glb`/`.gltf`	`AssetLoadSystem` читает Assimp и собирает единый массив вершин/индексов в `JobSystem`	`AssetLoadSystem::Update()` создаёт Direct3D-буферы на главном потоке	пока один упрощённый material slot и fallback-текстура
Корабль `SpaceShip.glb`	worker читает Assimp, строит список узлов, collision proxy и CPU-batch панелей	`SceneManager::UpdateAsyncShipImport()` добавляет узлы и batch-материалы	сохраняются встроенные материалы корабля

Что загружается	CPU-часть	Установка результата	Материалы
		порциями	

Это два конвейера, потому что корабль — не просто один render mesh. Имена его узлов создают комнаты, двери, панели корпуса, генераторы, свет, физику и связи AI.

Главное правило потоков

Worker thread может:

- читать файл;
- вызывать `Assimp::Importer::ReadFile`;
- обходить неизменяемый `aiScene`;
- считать вершины, индексы и таблицы;
- подготавливать обычные C++-контейнеры.

Только главный поток может:

- вызывать Direct3D через текущие `device/context`;
- менять `ECSRegistry`;
- создавать/удалять Bullet-тела;
- менять ImGui state;
- подключать готовый объект к сцене.

Если вызвать `ID3D11DeviceContext` или менять ECS из worker, можно получить редкий crash, гонку данных или повреждённую сцену. Поэтому «асинхронная загрузка» не означает «всё делать в другом потоке». Она означает: тяжёлая чистая CPU-подготовка идёт в фоне, а зависимые от главного потока шаги выполняются небольшими порциями.

Состояния обычной модели

`AssetState` проходит такую цепочку:

```
Unloaded -> Loading -> CpuLoaded -> Ready
                         |           |
                         +----------> Error
```

- `Loading`: worker читает файл и строит массивы.
- `CpuLoaded`: массивы готовы, но GPU ещё ничего о них не знает.
- `Ready`: vertex/index buffers созданы, renderer может привязать mesh.
- `Error`: Assimp или Direct3D вернул ошибку; текст виден в панели.

Запуск происходит без ожидания:

```
m_assetLoadSystem.RequestMeshLoad(req.modelPath);

// Entity появляется сразу с безопасной fallback-моделью.
Entity entity = m_sceneManager.SpawnObject(std::move(desc));
auto* mesh = m_sceneManager.GetObjectMesh(entity);
```

```
mesh->asyncModelPath = req.modelPath;
mesh->useAsyncMesh = true;
```

Каждый кадр главный поток забирает готовые задачи и создаёт GPU-ресурсы:

```
m_jobSystem.ExecuteMainThreadJobs();
m_assetLoadSystem.Update(m_direct3D->GetDevice());
```

Для статического GLB используется `aiProcess_PreTransformVertices`. Assimp заранее применяет трансформации узлов к вершинам, поэтому после объединения мешей отдельные детали не сваливаются в начало координат. Такой путь подходит для неподвижных декораций. Для скелетной анимации `PreTransformVertices` применять нельзя: кости и иерархию нужно загружать отдельным анимационным конвейером.

Почему GPU upload остаётся на главном потоке

`ID3D11Device` технически допускает часть многопоточной работы, но текущий renderer и lifetime ресурсов проекта построены вокруг одного потока. `ID3D11DeviceContext` особенно нельзя бездумно использовать параллельно. Безопасный первый вариант — оставить `CreateBuffer` на главном потоке.

Главное зависание раньше происходило на чтении и полном разборе тяжёлого файла. После переноса CPU-части возможен небольшой единичный скачок на создании огромного буфера, но окно продолжает обновляться во время чтения. Если один обычный mesh станет настолько велик, что и `CreateBuffer` заметно тормозит, следующий этап развития — делить CPU-геометрию на chunks, например по 100–250 тысяч вершин, и загружать один chunk за кадр.

Конвейер корабля

У корабля больше стадий:

```
Queued
  -> ReadingFile                worker
  -> PreparingCpuData           worker
  -> CpuReady
  -> UploadingNodes             main thread, порциями
  -> UploadingPanelBatches      main thread, не более одного batch за кадр
  -> Finalizing                 main thread
  -> Ready или Error
```

Запуск:

```
if (!m_sceneManager.HasImportedShip())
{
    const bool accepted =
        m_sceneManager.RequestShipImportAsync("assets/model/SpaceShip.glb");
}
```

Продолжение каждый кадр:

```
m_sceneManager.UpdateAsyncShipImport(
    m_direct3D->GetDevice(),
    m_direct3D->GetDeviceContext(),
    4.0f); // желаемый бюджет установки на один кадр, миллисекунды
```

Значение `4.0f` — не гарантия, что любой кадр займёт ровно четыре миллисекунды. Одна неделимая операция, например создание большого texture resource или Bullet triangle mesh,

может занять больше. Бюджет означает, что после законченного узла система проверит время и не начнёт ещё десяток узлов в том же кадре.

`Assimp::Importer` хранится внутри состояния загрузки до завершения импорта. Это обязательно: указатели `aiScene*`, `aiNode*` и `aiMaterial*` принадлежат importer. Нельзя создать importer как локальную переменную worker-функции, вернуть голый `aiScene*`, уничтожить importer и потом обращаться к этому указателю на главном потоке.

Упрощённая схема правильного lifetime:

```
struct AsyncState
{
    std::unique_ptr<Assimp::Importer> importer;
    const aiScene* scene = nullptr; // жив, пока жив importer
    std::vector<NodeTask> nodes;
};

auto state = std::make_shared<AsyncState>();
jobs.SubmitJob([state]
{
    state->importer = std::make_unique<Assimp::Importer>();
    state->scene = state->importer->ReadFile(path, flags);
    // Подготовить nodes, затем атомарно опубликовать CpuReady.
});
```

После `Finalizing` система:

1. связывает двери с комнатами;
2. связывает hull panels с их комнатами;
3. создаёт временный переход GUN ↔ CAPTAIN для AI;
4. очищает временные CPU-таблицы;
5. уничтожает importer;
6. публикует `Ready`.

Панель загрузки

`EditorUI::RenderAssetLoadingPanel()` автоматически появляется сверху окна, когда есть активная загрузка или ошибка. Она показывает:

- путь к файлу;
- текущую стадию;
- progress bar;
- число обработанных meshes для обычной модели;
- число установленных узлов/batches для корабля;
- текст ошибки.

Кнопка `SpawnShip` блокируется, пока корабль загружается, и после появления `ShipRoot`. Это защищает от двух параллельных импортов одного корабля.

Во время стадии `Reading GLB with Assimp` процент приблизительный. Assimp не даёт точный callback чтения GLB, поэтому честнее показать активную стадию и анимацию, чем изображать ложные «57%». После CPU-подготовки прогресс считается по реальным узлам и batches.

Ограничения текущей версии

1. Обычный статический GLB асинхронно объединяется в один mesh и временно получает fallback-материал. Встроенные PBR-материалы полностью сохраняет специализированный импорт корабля. Следующее расширение — `MeshCpuData` с массивом submesh/material ranges.
2. Отдельный collision mesh из Spawn panel пока загружается старым синхронным путём. Для теста чистой асинхронности выключай `Spawn with physics`; затем добавь отдельный async collision pipeline.
3. Assimp не умеет прервать `ReadFile` посередине безопасным простым флагом. Поэтому кнопка Cancel не добавлена: фальшивая отмена лишь спрятала бы панель, но worker продолжал бы занимать память и CPU.
4. Один особенно тяжёлый узел корабля всё ещё может дать короткий spike во время GPU/physics установки. Лечится предварительной CPU-конвертацией узлов и chunked upload, а не созданием ещё одного потока с `DeviceContext`.

Как проверить без догадок

1. Запусти Debug x64.
2. В Spawn укажи небольшой `.glb`, нажми `Spawn`.
3. Убедись, что панель появилась, камера и редактор продолжают отвечать.
4. Дождись `Ready`: fallback-модель должна замениться загруженной геометрией.
5. Перезапусти сцену, нажми `SpawnShip`.
6. Во время чтения корабля перемещай камеру и открывай панели — окно не должно получать статус «Не отвечает».
7. Проверь последовательность стадий и рост Items.
8. После исчезновения панели проверь комнаты, двери, коллизии, панели, свет и путь NPC.
9. Укажи несуществующий `.glb`: панель должна показать `Error`, а приложение не должно упасть.
10. Замерь самый длинный кадр отдельно на `ReadingFile` и на `UploadingNodes`. Это две разные проблемы.

Если панель замерла

Симптом	Вероятная причина	Что проверить
Долго стоит `Reading GLB`	Assimp читает 250-МБ файл или диск медленный	CPU/диск в диспетчере задач; существование и размер файла
`Reading GLB`, но окно отвечает	это не freeze, worker ещё занят	подождать; не нажимать Spawn

Симптом	Вероятная причина	Что проверить
		повторно
`Creating Direct3D buffers`, затем Error	не хватило VRAM/памяти или слишком большой buffer	`error`, D3D debug layer, число вершин
Корабль дошёл до узлов и упал	ошибка конкретного узла, материала или physics shape	последняя строка `Ship node:` в debugger output
Обычный GLB серый	ожидаемое ограничение fallback-материала	использовать ship pipeline либо реализовать material ranges
Детали обычного GLB разъехались	забыли `aiProcess_PreTransformVertices`	проверить flags `LoadMeshFromFile`
Сцена очищена во время загрузки	старый результат нельзя применять в новую сцену	`SceneManager::Clear()` сбрасывает ссылку на async state

10. Рендер DirectX 11 простыми словами

Упрощённая цепочка:

1. CPU обновляет данные сущностей;
2. `RenderSystem` собирает видимые render items;
3. модель предоставляет vertex/index buffers;
4. материал предоставляет текстуры и параметры;
5. vertex shader преобразует вершины;
6. pixel shader вычисляет цвет пикселя;
7. Direct3D записывает результат в render target;
8. кадр выводится на экран.

Основные понятия

- **Mesh** — геометрия: вершины и индексы.
- **Model** — один или несколько mesh и связанные данные импорта.
- **Material** — как поверхность выглядит.
- **Texture/SRV** — изображение, доступное шейдеру.
- **Shader** — программа GPU.
- **Constant buffer** — небольшие данные кадра/объекта для GPU.
- **Draw call** — команда нарисовать набор геометрии.

Почему нельзя хранить gameplay в renderer

Рендер может пропустить невидимый объект или работать иначе в headless-тесте. Состояние пожара должно существовать в ECS, а renderer только рисует пламя по этому состоянию.

Добавление визуального объекта

1. Импортируй/загрузи модель.
2. Создай сущность с `TransformComponent`.
3. Добавь `MeshRendererComponent` и материал.
4. Убедись, что объект активен.
5. Проверь bounds/culling.
6. Для физики отдельно добавь collider/rigid body.

Диагностика невидимого объекта

- существует ли entity;
- есть ли `TransformComponent` и `MeshRendererComponent`;
- активен ли объект;
- не нулевой ли scale;
- есть ли модель и материал;
- загружен ли shader;
- попадает ли объект во frustum;
- правильны ли world/view/projection;
- не оказался ли объект внутри камеры или за near/far plane.

11. Bullet Physics и типы тел

Static, dynamic, kinematic

- **Static**: масса 0, не двигается физикой; пол и стены.
- **Dynamic**: имеет массу, скорость и реагирует на силы.
- **Kinematic/character**: положение задаётся контроллером, но объект участвует в столкновениях.

Collision shape и render mesh — не одно и то же

Меш можно рисовать подробно, но коллизию делать простой. Простые формы стабильнее и дешевле.

Доступные формы проекта включают box, sphere, cylinder, capsule, plane, convex hull, triangle mesh, GImpact, VHACD и stair ramp.

Общий пример rigid body

```
RigidBodyComponent body;
body.mass = 20.0f;
body.shapeType = CollisionShapeType::Box;
// Остальные поля заполняй по фактическому определению Components.h.
registry.AddComponent<RigidBodyComponent>(entity, std::move(body));
```

После изменения размера коллизии обычно требуется rebuild: существующий Bullet shape сам не изменится.

Лучи

Raycast нужен для взаимодействия, оружия, ремонта и проверки земли.

```
PhysicsRaycastHit hit;
if (PhysicsRaycast::ClosestHit(from, to, hit))
{
    Entity target = hit.entity;
    XMFLOAT3 point = hit.position;
}
```

Используй реальные сигнатуры из `Physics/PhysicsRaycast.h`; пример показывает принцип.

Физические единицы

Придерживайся одной шкалы, предпочтительно 1 world unit = 1 метр. Тогда масса — кг, скорость — м/с, ускорение — м/с².

Нельзя смешивать transform без правила владельца

Для каждого физического объекта реши:

- ECS пишет transform в Bullet;
- Bullet пишет transform в ECS;
- либо controller пишет в Bullet, затем PhysicsSystem синхронизирует ECS.

Если два владельца каждый кадр исправляют друг друга, появляется дрожание.

12. Капсульный контроллер игрока и NPC

Игрок и NPC используют общую систему `CharacterControllerSystem`. Это правильно: исправление физики должно работать для обоих.

Геометрия капсулы

Для Bullet capsule:

```
totalHeight = cylinderHeight + 2 * radius
halfHeight  = totalHeight * 0.5
bottomY     = centerY - halfHeight
```

Модель ног и физический низ капсулы должны иметь осознанное смещение. В проекте для этого существуют автоматический center offset и `footClearance`.

Почему персонаж дрожал на земле

Фактическая проблема состояла из двух частей:

1. капсула в покое постоянно попадала в коррекцию контакта с полом;
2. render interpolation обновляла previous/current не на каждом реальном fixed-step, из-за чего модель визуально прыгала между состояниями.

Рабочая схема:

- надёжный ground ray;
- idle snap центра капсулы к `floorY + halfHeight + footClearance`;
- обнуление микроскорости в покое;
- разрешение Bullet sleep;
- увеличение `PhysicsWorld::m_stepIndex` на каждый выполненный фиксированный шаг;
- обновление previous/current только по факту нового physics step;
- визуальный mesh читает корректно интерполированный transform.

Почему в воздухе дрожания не было

В воздухе отсутствовал контакт с полом, поэтому solver не делал вертикальную коррекцию. Это важный диагностический признак: ошибка зависела от ground contact, а не от анимации.

Почему нельзя просто поднять капсулу

Если поднять только Bullet capsule, а mesh оставить на старом transform, физическое и визуальное тела расходятся. Персонаж выглядит провалившимся или парящим.

Правильный вариант: единый корневой transform + явный визуальный offset, который не участвует в физике.

```
const float meshY = capsuleCenterY - capsuleHalfHeight - visualFeetLocalY;
```

Step-up через пороги

Для низкого препятствия контроллер должен:

1. определить стенку впереди нижним лучом/sweep;
2. проверить свободное место на высоте шага;
3. найти верхнюю поверхность;

4. убедиться, что наклон допустим;
5. плавно изменить целевую высоту;
6. сохранить горизонтальное намерение движения.

NPC не должен уметь проходить порог только при беге. Высота шага и алгоритм step-up не должны зависеть от скорости так, чтобы ходьба теряла возможность пройти тот же геометрический порог.

Инварианты контроллера

- в покое скорость стремится к нулю;
- при `grounded` низ капсулы не находится глубоко под полом;
- mesh и capsule остаются согласованы;
- один physics step создаёт ровно одно новое физическое состояние;
- controller не телепортируется одновременно двумя системами;
- NPC и player проходят один набор тестов.

Регрессионный тест персонажа

Сделай отдельную тестовую комнату:

- ровный пол;
- пороги 0.05, 0.10, 0.20, 0.30 и 0.36 м;
- узкая дверь;
- наклоны 10°, 30°, 45° и 50°;
- движущаяся платформа;
- NPC следует за игроком ходьбой и бегом;
- 60 секунд стояния на месте.

Проверяй позицию центра и нижней точки капсулы в debug overlay.

13. Игрок: ввод, движение, камера и параметры

Поток данных

```
Input → CharacterMovementSystem → CharacterMovement
      → CharacterControllerSystem → Bullet
      → PhysicsSystem → Transform
      → Camera/Animation/Render
```

Управление

- `W/A/S/D` — движение;
- `Shift` — бег;

- `Space` — прыжок;
- `E` — взаимодействие;
- `H` — выстрел текущей тестовой системы оружия;
- `L` — удержание ремонта;
- `F3` — HUD/debug-переключатель, заданный в `GameHud`;
- `F5` — Editor/Play;
- `F1` — окно object debug;
- `F2` — освобождение/возврат курсора в Play;
- `F6` — тест разрезания меша.

Сверяй раскладку с `ApplicationClass.cpp` и конкретной системой: она может развиваться.

PlayerComponent

Хранит здоровье, выносливость и кислород вместе с максимумами. После изменения вызывай нормализацию или самостоятельно ограничивай значения.

```
if (auto* player = registry.GetComponent<PlayerComponent>(playerEntity))
{
    player->oxygen -= oxygenUsePerSecond * dt;
    player->Normalize();
}
```

Игрок не супергерой

Поддерживай это механиками:

- переносимый вес и замедление;
- боль, кровопотеря и падение эффективности;
- ограниченный кислород;
- необходимость инструментов и расходников;
- невозможность мгновенно чинить сложные системы;
- зависимость от экипажа и инфраструктуры.

Не делай трудность только через огромный запас здоровья у врагов.

14. Взаимодействия и события

`InteractionSystem` посылает луч из камеры и по `E` создаёт действие для сущности с подходящим компонентом.

Хорошая граница:

- raycast отвечает «на что смотрит игрок»;
- компонент отвечает «можно ли взаимодействовать»;

- событие отвечает «что произошло»;
- целевая система меняет состояние двери/устройства.

Пример нового интерактивного предохранителя

```
struct FuseBoxComponent
{
    bool tripped = false;
    float maxLoadMW = 2.0f;
};

void ToggleFuse(ECSRegistry& registry, Entity entity)
{
    if (auto* fuse = registry.GetComponent<FuseBoxComponent>(entity))
        fuse->tripped = !fuse->tripped;
}
```

Не привязывай всю логику к клавише E: NPC и автоматическая система должны уметь выполнить ту же операцию через команду/событие.

15. Анимация и визуальная модель

Анимация должна отображать состояние, а не владеть физикой.

Пример выбора:

```
if (!movement.grounded)
    Play("Jump");
else if (speed > runThreshold)
    Play("Run");
else if (speed > walkThreshold)
    Play("Walk");
else
    Play("Idle");
```

Root motion

Если клип содержит движение корня, а controller одновременно двигает капсулу, возникает двойное перемещение. Для текущей архитектуры безопаснее in-place анимации: физика двигает entity, а скелет показывает походку.

Диагностика дрожания

1. Отключи анимацию. Если дрожит — причина ниже, в transform/physics.
2. Подними capsule в воздух. Если дрожание исчезло — смотри ground contact.
3. Нарисуй collider и mesh origin.
4. Выведи raw physics transform, interpolated transform и render transform.
5. Проверь, нет ли двух записей в один transform за кадр.

16. NPC AI: слои и поток решений

Архитектура NPC уже разделена на уровни:

```
NpcCommand
   ↓
NpcUtilitySystem — выбирает, что сейчас важнее
   ↓
StateSystem — выполняет состояние/задачу
   ↓
NpcNavMeshSystem — строит/выдаёт маршрут
   ↓
CharacterMovement + CharacterFacing
   ↓
CharacterControllerSystem + Animation
```

Команда

`NpcCommandType` включает остановку, блуждание, движение к позиции/комнате, следование и ремонт.

```
NpcCommandSystem::FollowEntity(brain, playerEntity);
NpcCommandSystem::RepairEntity(brain, breachEntity);
```

Проверь точную сигнатуру в `NpcCommandSystem.h`.

Utility AI

Utility AI вычисляет полезность вариантов. Это не последовательность `if player near`.

```
float repairScore = severity * engineerSkill * toolAvailability;
repairScore *= (1.0f - normalizedDistance);

float fleeScore = danger * fear;
float restScore = fatigue * fatigue;
```

Выбирается высокий score, но нужны:

- hysteresis, чтобы задача не менялась каждый кадр;
- минимальное время удержания задачи;
- cooldown;
- reservation цели;
- недоступность задачи без инструмента/знания/пути.

StateSystem

StateSystem превращает цель в конкретные действия: построить путь, открыть дверь, двигаться к следующему полигону, остановиться на дистанции ремонта, применить прогресс.

Состояния

`NpcState` включает Idle, Wandering, MoveToPosition, MoveToRoom, Following, MoveToRepair, WaitingForDoor, Repairing, Stopped и Blocked.

Мозг NPC

`NpcBrainComponent` хранит текущее состояние, цель, маршрут, таймер repath, информацию о застревании, оценки utility и debug status. Это runtime-состояние агента.

Почему NPC может дрожать рядом с целью

- arrival distance меньше ошибки физики/пути;
- агент каждый кадр чередует «иди» и «стой»;
- target — движущийся player, repath слишком частый;
- controller и steering спорят;
- collision capsules толкают друг друга;
- facing вращает transform, а physics возвращает старое состояние.

Используй hysteresis:

```
if (following)
{
    if (distance > resumeDistance)
        wantsMove = true;
    else if (distance < stopDistance)
        wantsMove = false;
}
```

Где `resumeDistance > stopDistance`.

17. NavMesh, комнаты и двери

NavMesh описывает область, где может ходить центр агента с заданными radius/height/step/slope.

Параметры должны соответствовать controller

- agent radius не меньше рабочего радиуса капсулы с небольшим запасом;
- agent height соответствует полной высоте;
- agent max climb согласован с normal step height;
- max slope согласован с controller;
- дверной проём после erosion остаётся достаточно широким.

Ошибка `Detour could not project start/target`

Она означает, что начальную или целевую точку не удалось сопоставить с ближайшим полигоном NavMesh.

Проверь:

1. NavMesh комнаты построен;
2. NPC действительно относится к этой комнате;

3. Y точки близок к поверхности NavMesh;
4. extents поиска не слишком малы;
5. collider пола попал в геометрию сборки;
6. порог/дверь не разорвали связность;
7. start и target не внутри стены.

Двери

Дверь — одновременно:

- физическое препятствие;
- связь двух комнат;
- управляемое устройство;
- потребитель энергии;
- ограничение атмосферы/воды;
- часть AI route.

AI не должен просто проходить через закрытую дверь. Ему нужен переход: подойти → запросить открытие → подождать → проверить проём → продолжить → при необходимости закрыть.

Отладка пути

В окне NPC Debug доступны rebuild room NavMeshes, force repath, follow player и переход к целям. Включай draw NavMesh in world, смотри количество полигонов и next point.

18. Как добавить новую задачу NPC

Пример: тушение пожара.

1. Данные мира

```
struct FireComponent
{
    float intensity = 0.2f;
    float fuel = 10.0f;
    Entity room = InvalidEntity;
};

struct FireExtinguisherComponent
{
    float agentKg = 5.0f;
};
```

2. Команда и состояние

```
enum class NpcCommandType
{
    // существующие значения...
    ExtinguishFire
```

```
};

enum class NpcState
{
    // существующие значения...
    MoveToFire,
    Extinguishing
};
```

3. Utility score

```
float ScoreFireTask(const FireComponent& fire,
                    float distanceMeters,
                    bool hasExtinguisher,
                    float dangerTolerance)
{
    if (!hasExtinguisher)
        return 0.0f;

    const float urgency = std::clamp(fire.intensity, 0.0f, 1.0f);
    const float distancePenalty = 1.0f / (1.0f + distanceMeters * 0.1f);
    return urgency * distancePenalty * dangerTolerance;
}
```

4. Reservation

Без reservation десять NPC выберут один пожар.

```
struct TaskReservationComponent
{
    Entity reservedBy = InvalidEntity;
    float expiresAt = 0.0f;
};
```

5. Выполнение

1. Выбрать доступный огнетушитель.
2. Зарезервировать инструмент и пожар.
3. Построить маршрут.
4. Пройти двери.
5. Остановиться на безопасной дистанции.
6. Уменьшать intensity и запас огнетушителя.
7. Завершить/отменить задачу.
8. Освободить reservation при любом выходе.

6. Debug UI

Показывай score, выбранный fire entity, причину отказа, путь, reservation и время состояния.

7. Тесты

- один NPC и один пожар;
- два NPC и один пожар;
- огнетушителя нет;

- дверь закрыта и обесточена;
- пожар погас сам во время пути;
- NPC получил более срочную угрозу;
- цель уничтожена.

19. Энергетика

Основные данные

- `GeneratorComponent` — топливо, выход MW, ramp, нагрев, стабильность, SCRAM;
- `PowerConsumerComponent` — запрос энергии и приоритет;
- `PowerGridComponent` — генерация, спрос, батарея и brownout;
- `OxygenGeneratorComponent` — восстановление кислорода/давления/температуры;
- emergency battery — резерв при провале генерации.

Базовый баланс

```
available = generatorOutput + batteryDischarge
demand    = sum(activeConsumers)

if available >= demand:
    consumers powered
    excess may charge battery
else:
    disable/throttle lowest priority consumers
```

Приоритеты

Питание должно распределяться детерминированно:

1. управление реактором и безопасность;
2. гермодвери, насосы, жизнеобеспечение;
3. связь и навигация;
4. производство и комфорт;
5. декоративное освещение.

Это лишь стартовый дизайн; значения должны храниться в данных.

Пример потребителя

```
PowerConsumerComponent consumer;
consumer.requestedPowerMW = 0.35f;
consumer.priority = PowerPriority::LifeSupport;
registry.AddComponent<PowerConsumerComponent>(entity, consumer);
```

Используй фактические имена полей/enum из `PowerConsumerComponent.h`.

Не связывай систему с игроком

Генератор вырабатывает мощность независимо от расстояния до игрока. NPC может запустить его. Дверь может остаться без питания, даже если игроку неудобно.

Диагностика

- фактическая генерация и максимальная;
- суммарный запрос;
- потребление по каждому приоритету;
- заряд/разряд батареи;
- причина, почему конкретный потребитель не powered;
- события trip/SCRAM/brownout.

20. Комнаты, атмосфера, давление и затопление

Комната — симуляционная ячейка. Она хранит или связывает объём, кислород, давление, температуру, воду, соседей и отверстия.

Поток через соединение

Упрощённо:

```
flow ∝ openingArea × pressureDifference × coefficient
```

При переносе массы/кислорода нельзя создать ресурс из ничего. Что ушло из A, должно прийти в B или выйти наружу.

Дверь и пробоина

Обе являются соединениями, но:

- дверь управляема и имеет `openProgress`;
- пробоина связана с повреждением панели;
- закрытая дверь имеет малую/нулевую эффективную площадь;
- отремонтированная пробоина должна перестать участвовать в потоке.

Устойчивость симуляции

Большой `dt` может перенести больше газа/воды, чем есть в комнате. Ограничивай перенос доступным количеством.

```
float moved = requestedFlow * dt;
moved = std::min(moved, source.availableMass);
source.availableMass -= moved;
```

```
target.availableMass += moved;
```

Декомпрессия

`DecompressionSourceComponent` описывает источник тяги. Сила влияет на физические объекты, частицы и персонажей, но итоговая физика должна применяться централизованно.

Тест сохранения величин

В закрытой системе сумма кислорода/массы до и после 10 минут симуляции должна почти совпадать. Допустимая погрешность должна быть измерена, а не оценена глазами.

21. Повреждения, оружие, разрезание и ремонт

Поток повреждения

```
Weapon/Raycast
  → Damage event
  → DamageSystem
  → health / panel damage / breach
  → RepairableComponent
  → particles + decal + decompression
  → player or NPC repair
  → mesh restoration + sealed breach
```

Оружие

Тестовая система стреляет по `H`. Raycast определяет hit entity/position/normal, затем damage system решает, что именно повредить.

Пробоина

Пробоина — отдельная entity или связанный runtime-объект с:

- ссылкой на панель;
- геометрией/операцией cut;
- состоянием sealed;
- параметрами потока;
- `RepairableComponent`;
- визуальным эффектом.

Ремонт

Игрок удерживает `L`, луч выбирает цель и `RepairSystem::ApplyRepairProgress` увеличивает прогресс. NPC использует тот же смысл операции, а не эмулирует клавишу.

```
const float progressDelta = repairSpeedPerSecond * dt;
RepairSystem::ApplyRepairProgress(registry, target, progressDelta, &jobs);
```

Проверь реальную сигнатуру перед вставкой.

Асинхронный mesh repair

Тяжёлое восстановление runtime mesh может выполняться через JobSystem. Пока задача идёт:

- entity не должна быть уничтожена без отмены/проверки;
- worker не должен трогать D3D immediate context;
- результат применяется на main thread;
- UI показывает состояние in progress;
- повторный ремонт не создаёт вторую одинаковую job.

Инварианты ремонта

- progress ограничен `[0, maxProgress]`;
- завершение вызывается один раз;
- пробоина после завершения sealed;
- поток прекращается;
- decal удаляется;
- panel breach count обновляется;
- runtime mesh восстановлен;
- ссылки не указывают на уничтоженную entity.

22. Корабль, плавучесть, давление корпуса и соединения

Корабль — иерархия сущностей: root, части, панели, комнаты и joints.

Центр масс

Неправильный центр масс даёт странное вращение. Массу и положение частей нужно сводить в общий центр:

```
centerOfMass = Σ(position_i × mass_i) / Σ(mass_i)
```

Плавучесть

Сила Архимеда зависит от вытесненного объёма жидкости. В игровом приближении точки плавучести могут быть распределены по корпусу.

```
buoyancyForce = fluidDensity × displacedVolume × gravity
```

Затопление

Вода увеличивает массу и меняет центр масс. Поэтому затопленная носовая комната должна физически наклонять корабль, а не только менять UI.

Joints

Joint соединяет физические части и имеет ограничения. После разрушения connection system должна обновить логическую структуру, физику и связанные ссылки.

Отладка

Рисуй center of mass, точки сил, AABB, joints, величины силы и массу воды. Иначе настройка превращается в угадывание.

23. Частицы, взрывы и эффекты

Эффект отображает событие, но не является источником истины.

```
Explosion gameplay event
  ├─ radial damage
  ├─ impulse
  ├─ fire/decompression state
  ├─ particle emitter
  ├─ light flash
  └─ sound request
```

Lifetime

Каждый временный эффект должен иметь понятный конец. Иначе эмиттеры и сущности копятся.

Object pool

Для частых частиц выгоднее переиспользовать память, чем создавать/удалять тысячи объектов.

Детерминизм

Если случайность влияет на gameplay, храни seed. Визуальные частицы могут использовать отдельный случайный поток, чтобы не менять игровую симуляцию.

Подробности находятся в `PARTICLES_EXPLOSIONS_DECOMPRESSION_GUIDE_RU.md`.

24. Редактор, UI и диагностика

Редактор — часть движка, а не временная игрушка. Для каждой сложной системы добавляй наблюдаемость.

Что должно быть видно

- entity ID и Name;
- присутствующие компоненты;
- текущее состояние и цель;
- raw и derived значения;
- причина отказа;
- кнопка безопасного rebuild/repath;
- графика в мире: лучи, bounds, путь, контакты.

Правило runtime/rebuild

Если UI меняет размер Bullet shape или структуру NavMesh, поле в C++ меняется сразу, но внешний ресурс нужно явно пересобрать. Подписывай это в UI: «требуется Rebuild».

Debug UI не должен менять релизную логику

Кнопка `Follow player` выдаёт обычную `NpcCommand`, а не включает отдельный секретный режим AI. Тогда отладка проверяет настоящий pipeline.

Полезный диагностический шаблон

```
if (ImGui::TreeNode("Heat Debug"))
{
    ImGui::Text("Temperature: %.2f C", heat.temperatureC);
    ImGui::Text("Dissipation: %.3f/s", heat.dissipationPerSecond);
    ImGui::Text("State: %s", heat.temperatureC > 100.0f ? "Danger" : "Normal");
    ImGui::TreePop();
}
```

25. Безопасный рецепт добавления новой функции

Используй этот порядок почти всегда.

Шаг 1. Описать игроку невидимые правила

Пример: «Устройство нагревается, когда потребляет энергию; остывает в комнате; при перегреве ломается».

Шаг 2. Записать инварианты

- температура не NaN;
- сломанное устройство не работает;
- тепло зависит от времени, а не FPS;
- состояние сохраняется;

- NPC может обнаружить перегрев без игрока.

Шаг 3. Создать компонент данных

```
struct DeviceHeatComponent
{
    float temperatureC = 20.0f;
    float safeTemperatureC = 80.0f;
    float failureTemperatureC = 140.0f;
    float heatPerMW = 3.0f;
    bool overheated = false;
};
```

Шаг 4. Создать одну систему

```
class DeviceHeatSystem
{
public:
    void Update(ECSRegistry& registry, float dt)
    {
        registry.ForEach<DeviceHeatComponent, PowerConsumerComponent>(
            [dt](Entity,
                 DeviceHeatComponent& heat,
                 PowerConsumerComponent& power)
            {
                const float generated = power.actualPowerMW * heat.heatPerMW;
                const float cooling = (heat.temperatureC - 20.0f) * 0.08f;
                heat.temperatureC += (generated - cooling) * dt;
                heat.temperatureC = std::max(0.0f, heat.temperatureC);
                heat.overheated = heat.temperatureC >= heat.safeTemperatureC;
            });
    }
};
```

Адаптируй имя `actualPowerMW` к фактическому компоненту.

Шаг 5. Вставить систему в явное место кадра

После расчёта power grid, но до систем, которые используют результат перегрева.

Шаг 6. Добавить debug UI

До красивых эффектов выведи числа и состояние.

Шаг 7. Добавить сохранение

Сохраняй авторитетные данные; производные флаги можно пересчитать, но только если это явно сделано после load.

Шаг 8. Добавить последствия

Сначала поломка и отключение, затем частицы/звук, затем AI-задача.

Шаг 9. Написать тестовую сцену

- нормальная мощность;

- перегрузка;
- отключение питания;
- отсутствие охлаждения;
- save/load в момент перегрева.

Шаг 10. Только потом расширять

Не начинай одновременно пожар, температуру комнат, новые материалы и 12 NPC-профессий. Сначала одна завершённая вертикаль.

26. Диагностика: от симптома к причине

Игра не компилируется

1. Найди первую ошибку.
2. Открой файл и строку.
3. Проверь include, namespace, сигнатуру и тип.
4. Если `unresolved external`, реализация не попала в build или сигнатуры различаются.
5. Если ошибка после добавления файла, проверь `.vcxproj`/Solution Explorer.

Игра падает

1. Запусти под debugger.
2. Смотри Call Stack.
3. Проверь `nullptr` и валидность entity.
4. Проверь время жизни ресурсов.
5. Временно добавь assert перед опасной операцией.

```
assert(entity != InvalidEntity);
assert(transform != nullptr);
```

Объект дрожит

- два владельца transform;
- physics/render работают с разными временными состояниями;
- penetration и solver correction;
- target steering без hysteresis;
- анимационный root motion;
- неправильный fixed-step/interpolation.

NPC не идёт

- команда не выдана;
- utility выбрал другую задачу;
- StateSystem не заполняет movement;
- start/target не проецируются на NavMesh;
- путь пуст/заблокирован дверью;
- controller grounded/step-up не работает;
- arrival distance уже считает цель достигнутой.

NPC застрял на пороге

- climb NavMesh и step height controller различаются;
- верхний clearance sweep упирается в геометрию;
- маленькая скорость не проходит ветку step-up;
- капсула слишком широка;
- пол и порог имеют щель/перекрытие;
- path corner расположен внутри препятствия.

Значение зависит от FPS

Скорее всего, забыто умножение на `dt` или операция ошибочно выполняется и на frame update, и на fixed update.

После load всё сломано

- не создан runtime physics/D3D ресурс;
- entity link не прошёл remap;
- новое поле не имеет default;
- система использовала данные до post-load fixup;
- файл сцены был записан частично.

Утечка или постепенное замедление

- временные entities не уничтожаются;
- GPU resource создаётся каждый кадр;
- вектор/очередь событий не очищается;
- JobSystem копит завершённые задачи;
- debug draw накапливает примитивы;
- particle emitters не завершаются.

27. Производительность без преждевременной оптимизации

Сначала измеряй.

CPU

- время всего кадра;
- время каждой крупной системы;
- число сущностей в обходе;
- число physics bodies и contacts;
- число NavMesh repath в секунду;
- время JobSystem queue/wait.

GPU

- draw calls;
- triangles;
- shadow passes;
- размер render targets;
- overdraw частиц;
- количество смен shader/material.

Полезные стратегии

- spatial culling и frustum culling;
- LOD;
- batching одинаковых панелей;
- не строить путь NPC каждый кадр;
- фиксировать симуляцию далёких зон с более редким tick;
- тяжёлую CPU-геометрию отдавать в JobSystem;
- D3D resource creation/apply оставлять main/render thread.

Мир не должен останавливаться вне камеры

Можно снижать частоту обновления, но состояние должно эволюционировать. Например, далёкая комната обновляет температуру раз в секунду интегральным шагом, а не каждый кадр.

28. Save game и персистентный мир

Сцена редактора и сохранение игры — разные понятия.

- Scene data: исходное расположение и настройки.
- Runtime save: здоровье, поломки, состояние сети, задачи, время мира.

Версия формата

```
{
  "saveVersion": 3,
  "worldTimeSeconds": 18342.5,
  "entities": []
}
```

При изменении схемы либо поддерживай defaults, либо миграцию.

Что не сохранять напрямую

- указатели;
- D3D resources;
- Bullet objects;
- открытые файлы;
- lambda/callback;
- временные debug данные.

Сохраняй описание, затем восстанавливай runtime-ресурс.

Атомарная запись

Безопасная схема: записать во временный файл, проверить, затем заменить основной. Не перезаписывай единственный save, пока новый JSON не записан полностью.

29. Архитектура живого мира

Симуляция должна отвечать на пять вопросов

1. Какие ресурсы существуют?
2. Кто их производит и потребляет?
3. Какие агенты знают о событиях?
4. Какие задачи можно создать?
5. Какие последствия происходят без участия игрока?

Perception и knowledge

NPC не должен знать всё. Разделяй факт мира и знание NPC.

```
struct NpcMemoryEntry
{
    Entity subject = InvalidEntity;
    XMFLOAT3 lastKnownPosition{};
```

```
    float observedAt = 0.0f;
    float confidence = 1.0f;
};
```

Пожар существует в мире, но NPC узнаёт о нём через зрение, датчик, сигнализацию или сообщение другого NPC.

Job board

События создают задачи мира: ремонт, лечение, доставка, тушение, эвакуация.

```
enum class WorldTaskType { Repair, Extinguish, Rescue, Haul };

struct WorldTask
{
    uint64_t id = 0;
    WorldTaskType type{};
    Entity target = InvalidEntity;
    float priority = 0.0f;
    Entity reservedBy = InvalidEntity;
};
```

NPC оценивает доступные задачи с учётом роли, навыка, риска, расстояния, инструмента и текущей обязанности.

Incident director без вращения мира вокруг игрока

Director может создавать события на основании состояния мира, но не обязан выбирать комнату игрока. Лучше учитывать:

- износ;
- перегрузку;
- обслуживание;
- качество деталей;
- историю предыдущих отказов;
- редкую случайность с фиксированным seed.

Fairness

Случайность должна иметь причины и предупреждения. Игрок может не суметь предотвратить событие, но должен иметь шанс понять, почему оно произошло.

30. Правила качества проекта

Definition of Done для механики

- она компилируется без новых ошибок;
- работает при 30, 60 и нестабильном FPS;
- не требует присутствия игрока, если это мировая система;

- имеет debug output/UI;
- сохраняется или осознанно является временной;
- обрабатывает уничтожение цели;
- имеет тестовую сцену;
- не создаёт ресурс каждый кадр;
- документация обновлена;
- старые ключевые сценарии не сломаны.

Имена

- компонент: существительное + `Component`;
- система: действие/область + `System`;
- булево: `is/has/can/should` или ясное состояние;
- единицы в имени: `distanceMeters`, `powerMW`, `durationSeconds`;
- функции-команды: `Start`, `Stop`, `Request`, `Apply`, `Rebuild`.

Комментарии

Комментируй причину и ограничение, а не очевидную операцию.

```
// Хорошо: предотвращает переключение Follow/Idle на границе дистанции.
const float resumeDistance = stopDistance + 0.5f;
```

Маленькие коммиты

Один коммит — одна понятная причина изменения. Пример: `Fix physics interpolation step tracking`, а не `many fixes`.

31. Что развивать следующим

Приоритетная цепочка для концепта игры:

1. регрессионные тесты и телеметрия существующих систем;
2. единое simulation time и события;
3. ограниченное perception/memory NPC;
4. world task board и reservation;
5. пожар/температура как вертикальная авария;
6. NPC, который сам обнаруживает и тушит пожар;
7. роли, навыки, усталость и травмы экипажа;
8. персистентный save мира;
9. один цельный аварийный сценарий;
10. баланс, звук и читаемость.

Эта последовательность развёрнута в `ROADMAP_14_DAYS_OFFLINE_STUDY_RU.md`.

Приложение A. Краткая карта компонентов

Компонент/группа	Назначение
`TransformComponent`	позиция, поворот, масштаб
`NameComponent`	читаемое имя
`ActiveComponent`	включение сущности
`MeshRendererComponent`	визуальная модель/материал
`RigidBodyComponent`	Bullet rigid body и shape
`CharacterController`	Bullet ghost/capsule/controller
`CharacterMovement`	намерение движения/прыжка
`PlayerGameTag`	помечает активного игрока
`PlayerComponent`	health/stamina/oxygen
`NPCComponent`	базовые данные NPC
`NPCMovementComponent`	скорости и дистанции
`NpcBrainComponent`	состояние, цель, путь, utility/debug
`RoomComponent`	помещение и связи
`DoorComponent`	дверь, комнаты, питание, поток
`CompartmentComponent`	физическая/атмосферная секция
`PowerGridComponent`	баланс сети и батареи
`GeneratorComponent`	генерация энергии
`PowerConsumerComponent`	запрос и приоритет энергии
`OxygenGeneratorComponent`	восстановление среды
`DamageableComponent`	получение урона
`HullPanelComponent`	панель корпуса
`HullBreachComponent`	пробоина
`RepairableComponent`	прогресс ремонта
`RuntimeMeshDamageComponent`	операции повреждения меша
`DecompressionSourceComponent`	источник декомпрессионной силы

Компонент/группа	Назначение
`SubmarineComponent`	состояние корабля/подлодки
`BuoyancyComponent`	параметры плавучести

Приложение B. Чек-лист перед поездкой

- Visual Studio запускается без сети;
- solution и зависимости уже скачаны;
- Debug и Release x64 собираются;
- сохранена рабочая сцена и её резервная копия;
- документы открываются в Markdown и DOCX;
- рядом лежит этот справочник и roadmap;
- есть локальная ветка/коммит «до поездки»;
- известен путь к exe;
- test room содержит пол, пороги, двери, NPC, генератор и пробоину;
- есть блокнот `DEV_NOTES_OFFLINE.md` для ежедневных итогов.

Приложение C. Шаблон записи о баге

```
Название:
Дата и commit:
Сцена:
Шаги воспроизведения:
Ожидание:
Фактический результат:
Частота: всегда / иногда / один раз
Зависит от FPS:
Зависит от контакта/комнаты/двери:
Какие debug значения видел:
Минимальное изменение, после которого появилось:
Гипотеза:
Проверка гипотезы:
Итоговая причина:
Исправление:
Регрессионный тест:
```

Приложение D. Словарь

- **AABB** — ограничивающий параллелепипед без поворота.
- **Accumulator** — накопленное время до следующего fixed-step.
- **Collider** — упрощённая геометрия столкновения.
- **Component** — данные одной стороны сущности.
- **Draw call** — команда GPU нарисовать геометрию.

- **ECS** — Entity Component System.
- **Entity** — ID объекта мира.
- **Fixed timestep** — постоянный шаг физической симуляции.
- **Hysteresis** — разные пороги включения и выключения состояния.
- **Interpolation** — плавное отображение между физическими состояниями.
- **Invariant** — условие, которое всегда должно оставаться истинным.
- **NavMesh** — сетка областей, доступных агенту.
- **Penetration** — взаимное проникновение коллайдеров.
- **RAII** — ресурс живёт столько же, сколько владеющий им объект C++.
- **Raycast** — поиск пересечения луча с миром.
- **Reservation** — временное закрепление задачи/ресурса за агентом.
- **Runtime resource** — объект Bullet/D3D, который создаётся при запуске и не сериализуется как указатель.
- **Solver** — часть физики, исправляющая контакты и ограничения.
- **System** — логика, обрабатывающая компоненты.
- **Utility AI** — выбор действия по численной полезности.
- **Vertical slice** — одна механика, пройденная от данных до UI, AI, сохранения и теста.

Конец справочника.